Large-Scale and Multi-Structured Databases

Project Design

SkyGraph

Luca Granucci

Lorenzo Marranini

Paolo Walsh

https://github.com/lorenzo-marranini/SkyGraphProject

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

UNIVERSITÀ DI PISA

CROSSLAB Innovation for industry 4.0

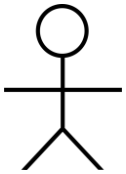
Application Highlights

SkyGraph is an advanced air traffic platform combining real-time flight tracking with historical data analysis.

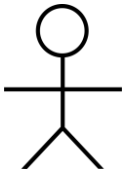
Main Features:

- An integrated system for real-time air traffic visualization with an interactive map and historical flight analysis.
- Calculation of performance metrics (delays, traffic volumes, distances) on airlines, airports, and specific routes, filterable by time ranges.
- Tools that allow airline companies to evaluate new routes based on shortest path algorithms and hub connectivity ratings.
- Analysis of airport statistics and visualization of emergency landings for airport flow management.

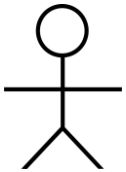
Actors and main mock-ups (i)



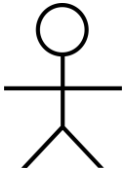
Guest *(non registered user)*



Traffic Controller *(registered user)*

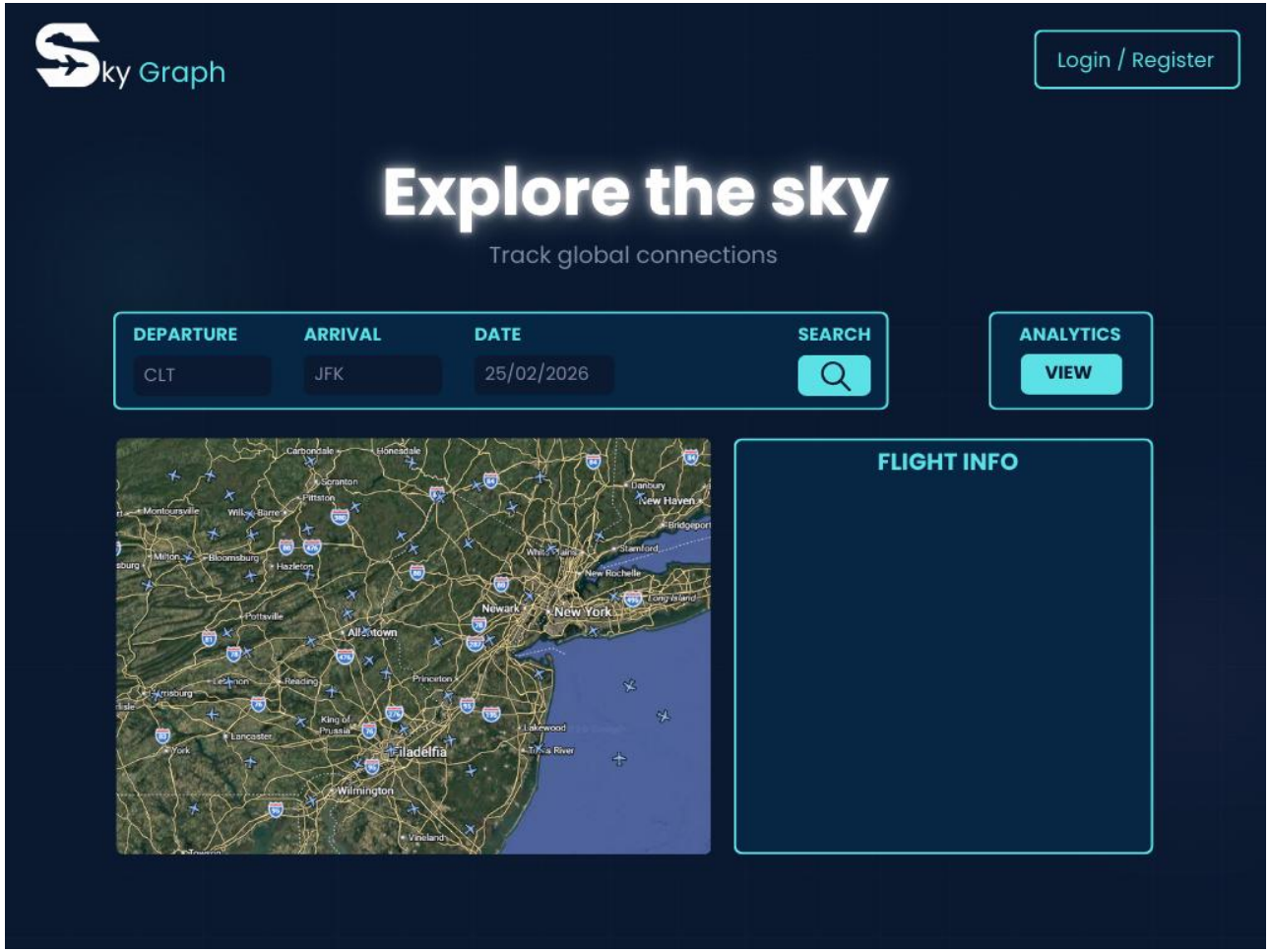


Airline Representative *(registered user)*



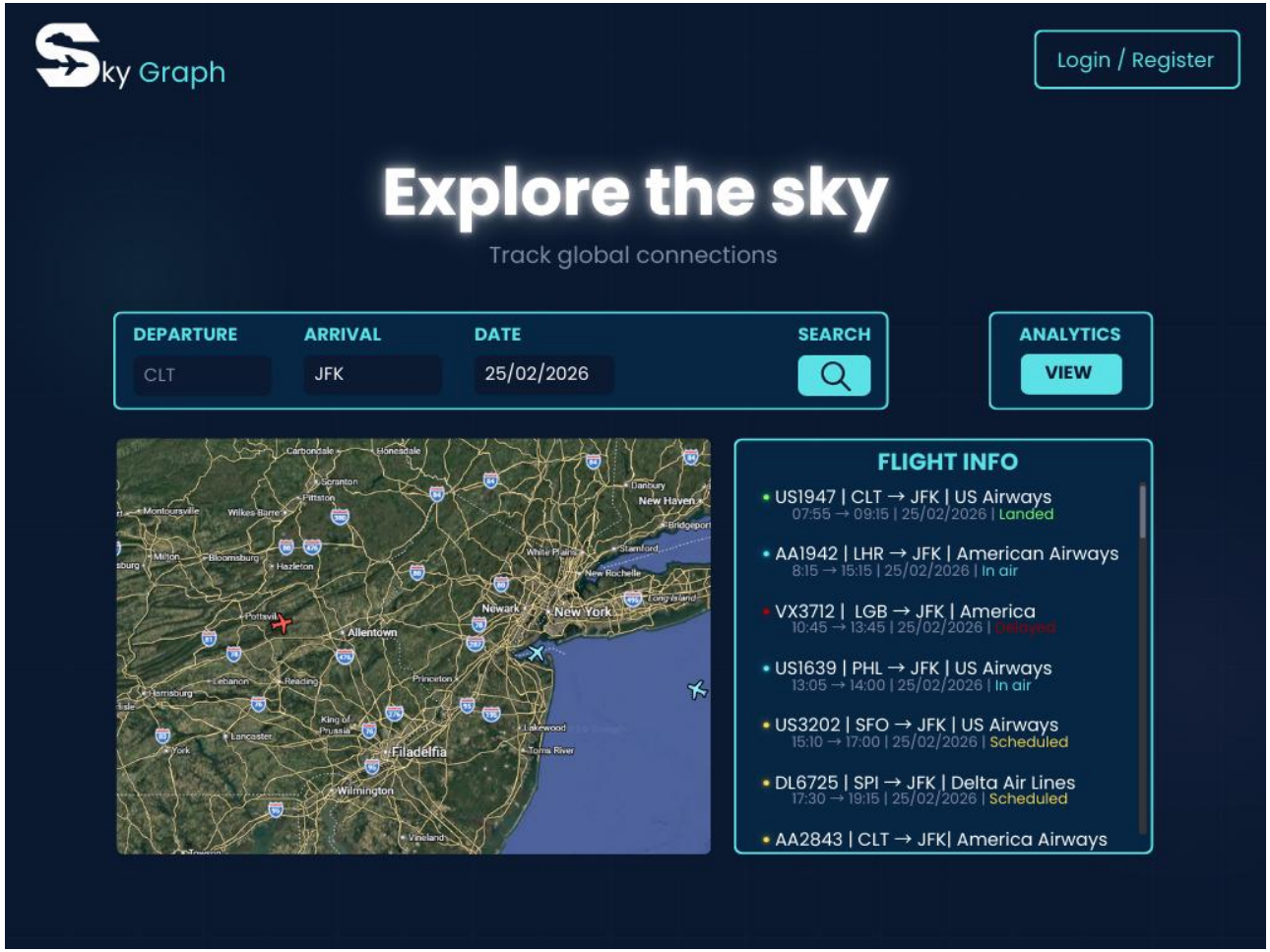
Administrator *(registered user)*

Actors and main mock-ups (ii)



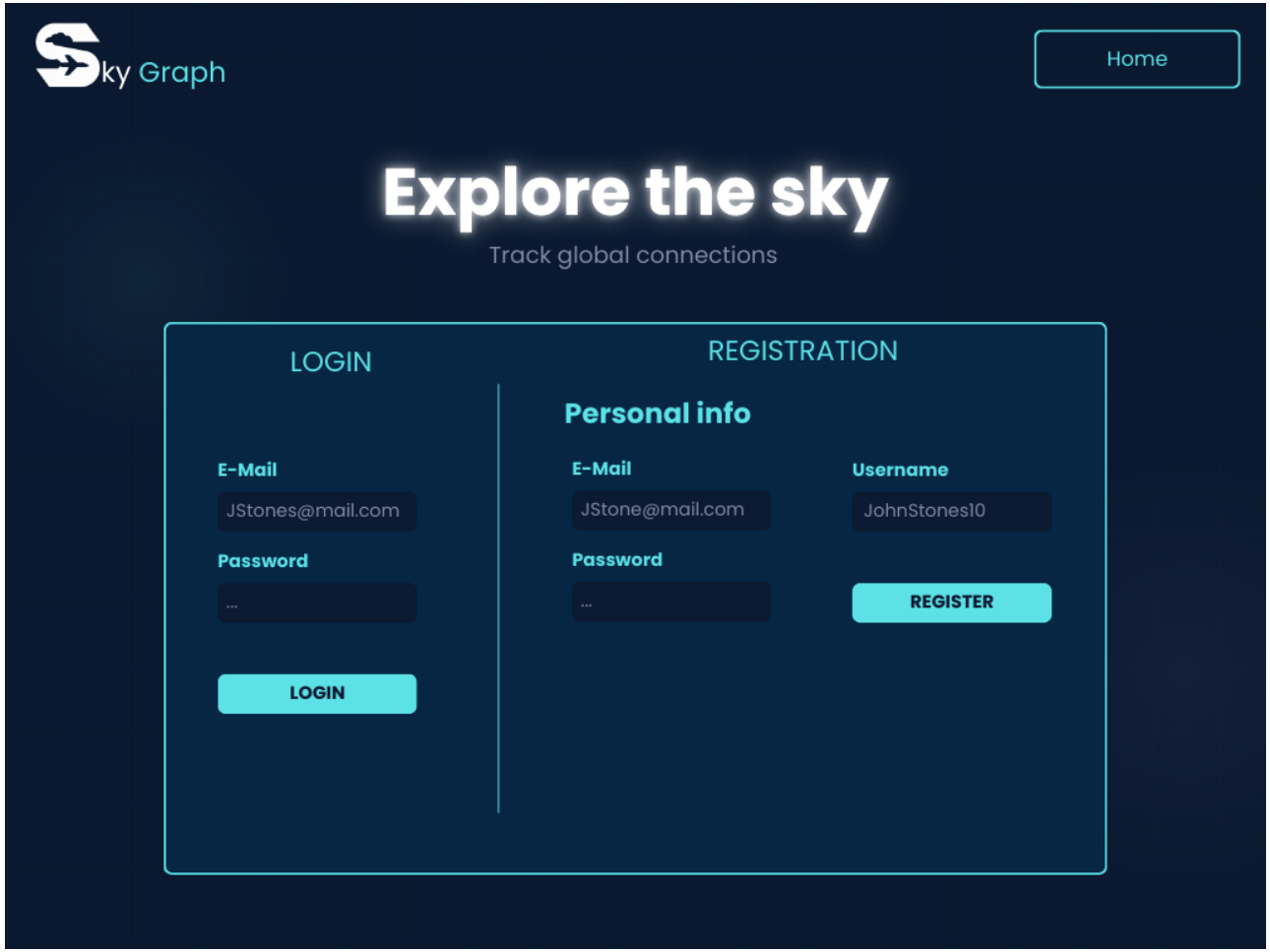
The main page

Actors and main mock-ups (iii)



Example of a search with applied filters

Actors and main mock-ups (iv)



The login/registration page

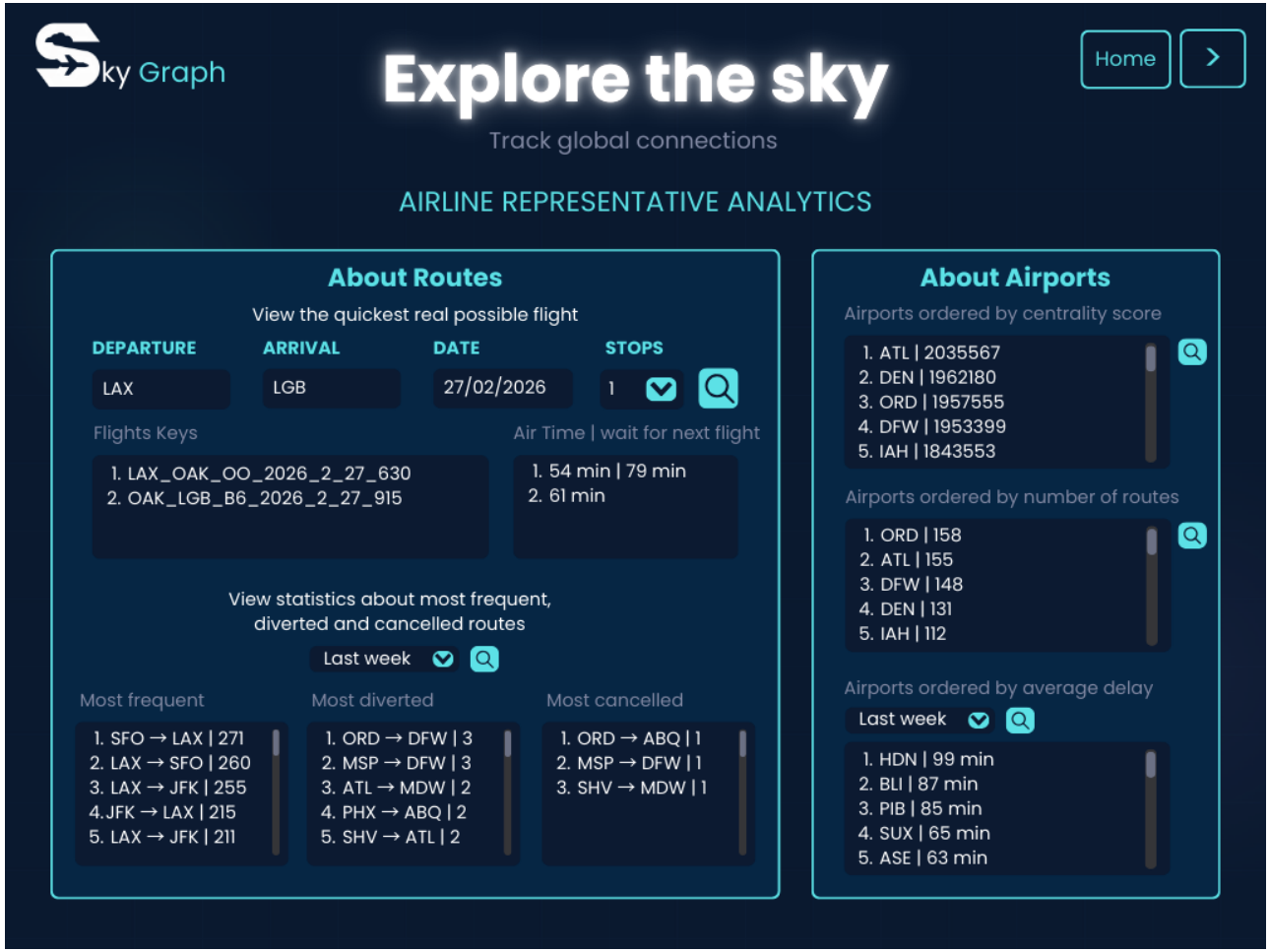
Actors and main mock-ups (v)

The Traffic Controller analytics 1/2

Actors and main mock-ups (vi)

The Traffic Controller analytics 2/2

Actors and main mock-ups (vii)



The Airline Representative analytics page 1/2

Actors and main mock-ups (viii)

The Airline Representative analytics page 1/2

Dataset Description

Sources: FlightRadar24 API and Kaggle Datasets about 2015 USA flights and USA cities.

Description: Data concerning live flights (FlightRadar24 API), historical flight data airports and airlines (Kaggle Datasets)

Volume: 4.7 million flights, 6500 flight logs, 311 cities, 342 airports

Variety: The final dataset is obtained by combining the datasets retrieved from Kaggle and real-time data from FlightRadar24 API.

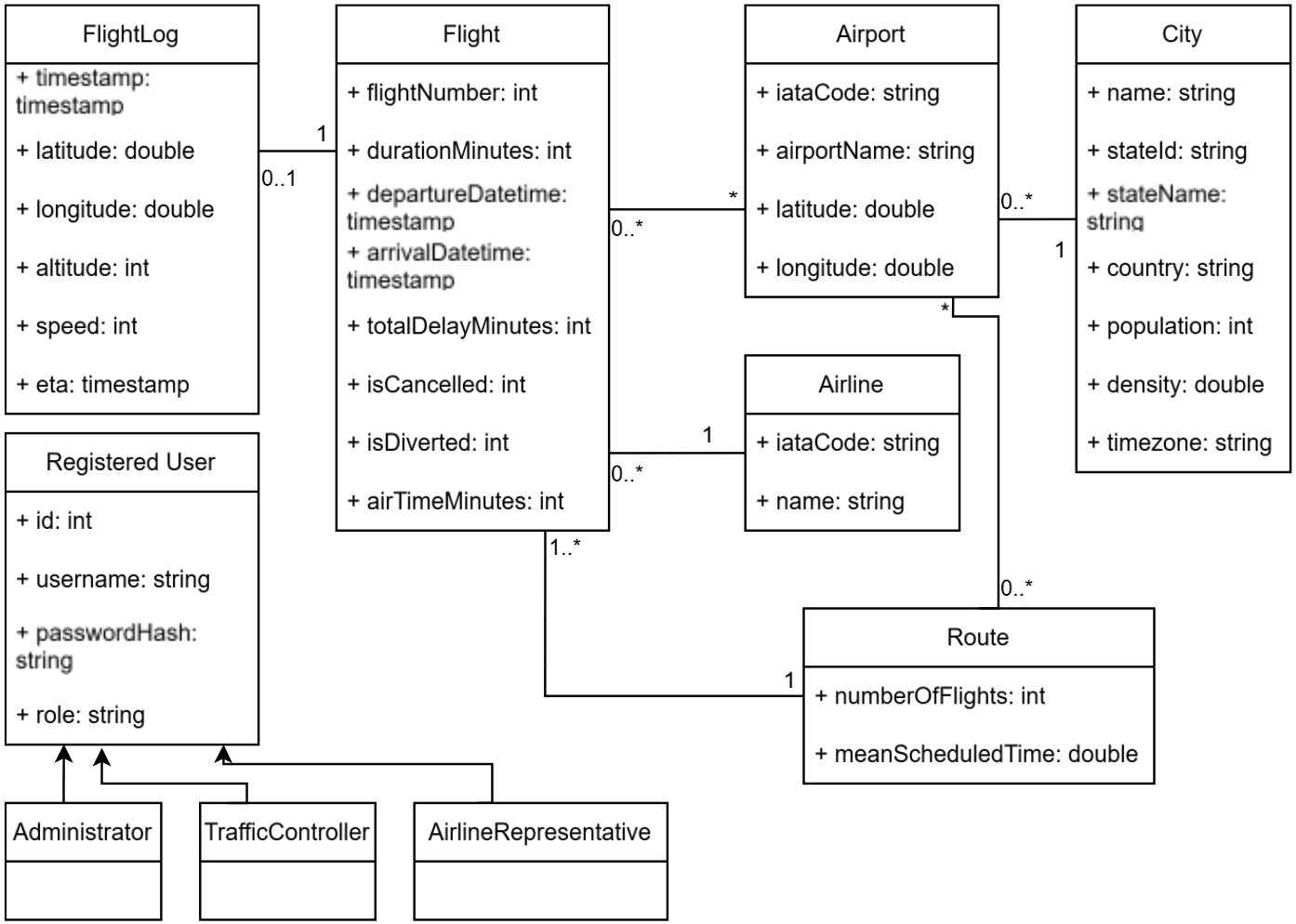
Velocity/Variability: Data about live position of flights is quickly outdated and overwritten.

Dataset Description

Pre-processing:

- Historical flight data was adjusted to establish a realistic timeline consisting of 70% past and 30% future flights.
- Live flight logs and summaries were harvested via the FlightRadar24 API, specifically targeting the US airspace.
- The retrieved live data structures were adapted to merge into the primary historical collection.
- Origin cities and airports for international flights entering the US were appended to the initial dataset.

UML Class Diagram



Application non-functional requirements

Product Requirements
1. The system must ensure low latency and must be highly available.
2. The system should be able to scale to accommodate increases in traffic and data load.

Implementation Requirements
1. The system must be developed using Object-Oriented Programming languages.
2. The system must expose its functionalities via RESTful APIs, documented according to the OpenAPI Specification.
3. The system must utilize a NoSQL database structure to efficiently handle large volumes of data.

Security Requirements
1. The system must implement a secure authentication protocol and enforce Role-Based Access Control to ensure users only access data pertinent to their role.
2. Data Protection: The system must encrypt users' passwords using hashing algorithms.

Document DB Design

Collections:

Flight Collection

User Collection

Airport Collection

Document DB Design

Queries:

Average delay per day of the week

```
@Aggregation(pipeline = {  1 usage   Lorenzo Marranini
        "{ '$match': { " +
                "'flight_info.schedule.departure_datetime': { '$gte': ?0, '$lte': ?1 }, " +
                "'stats.tot_delay_minutes': { '$ne': null } " +
                "} }",
        "{ '$project': { " +
                "'dayOfWeek': { '$dayOfWeek': '$flight_info.schedule.departure_datetime' }, " +
                "'delay': '$stats.tot_delay_minutes' " +
                "} }",
        "{ '$group': { " +
                "'_id': '$dayOfWeek', " +
                "'avgDelay': { '$avg': '$delay' } " +
                "} }",
        "{ '$sort': { 'avgDelay': -1 } }",
        "{ '$project': { " +
                "'_id': 0, " +
                "'dayOfWeek': '$_id', " +
                "'avgDelay': 1 " +
                "} }"
})
List<DayStatsDTO> findDaysByAvgDelay(Instant start, Instant end);
```

Nearest airport for emergency landing

```
@Aggregation(pipeline = {  1 usage   Lorenzo Marranini
        """
        {
            $geoNear: {
                near: { type: 'Point', coordinates: [ ?0, ?1 ] },
                key: 'location',
                distanceField: 'distanceKm',
                spherical: true,
                distanceMultiplier: 0.001
            }
        }
        """,
        "{ $limit: 5 }",
        """
        {
            $project: { _id: 1, name: 1, city: 1, country: 1, distanceKm: 1 }
        }
        """
})
List<AirportEmergencyDTO> findNearestAirports(double longitude, double latitude);
```

Document DB Design

Generate Airline Report

```
@Aggregation(pipeline = {  1 usage   Lorenzo Marranini
        "{ '$match': { " +
                "'flight_info.schedule.departure_datetime': { '$gte': ?0, '$lte': ?1 }, " +
                "'flight_info.airline.iata': ?2 " +
                "} }",
        "{ '$group': { " +
                "'_id': '$flight_info.airline.iata', " +
                "'totalKm': { '$sum': '$route.distance_km' }, " +
                "'avgKm': { '$avg': '$route.distance_km' }, " +
                "'totalFlights': { '$sum': 1 }, " +
                "'avgDelay': { '$avg': '$stats.tot_delay_minutes' }, " +
                "'totalDuration': { '$sum': '$stats.air_time_minutes' }" +
                "} }",
        "{ '$project': { " +
                "'_id': 0, " +
                "'airlineIata': '$_id', " +
                "'totalKm': 1, " +
                "'avgKm': 1, " +
                "'avgDelay': 1, " +
                "'totalFlights': 1, " +
                //calcolo efficiency
                "'efficiency': { " +
                "'$cond': [ " +
                "{ '$eq': ['$totalDuration', 0] }, " + // Se la durata è 0, evita divisione per zero
                "0, " +
                "{ '$divide': ['$totalKm', '$totalDuration'] }" +
                "] " +
                "}" +
                "} }"
})
List<AirlineReportDTO> generateAirlineReport(Instant start, Instant end, String AirlineIata);
```

MongoDB Replica Set Configuration

The MongoDB database of the application is deployed as a replica set consisting of three nodes across distinct Virtual Machines.

CAP Theorem:

Write:

- Flight Logs: write concern W1 policy. (AP)
- Other operations: write concern majority policy. (CP)

Read:

- All operations**:** nearest read policy. (AP)

MongoDB Replica Set Configuration

Implementation of write concern W1 only for Flight Logs

```java
public class FlightRepositoryCustomImpl implements FlightRepositoryCustom {  no usages  PaoloWalsh
    @Autowired  2 usages
    private MongoTemplate mongoTemplate;

    // custom update method to set the flight log for a specific flight key
    // uses write concern W1 to ensure availability while allowing for eventual consistency
    @Override  1 usage  PaoloWalsh
    public void updateFlightLogByKey(String flightKey, FlightMongo.FlightLog log) {
        Query query = new Query(Criteria.where( key: "flight_info.flight_key").is(flightKey));

        mongoTemplate.execute(FlightMongo.class, MongoCollection<Document> collection -> {
            var converter = mongoTemplate.getConverter();

            Document logDocument = new Document();
            converter.write(log, logDocument);

            Document queryDoc = query.getQueryObject();
            Document updateDoc = new Document("$set", new Document("flight_log", logDocument));

            collection.withWriteConcern(WriteConcern.W1)
                    .updateOne(queryDoc, updateDoc);

            return null;
        });
    }
}
```

MongoDB Indexes

Flight Collection:

- Flight_info.schedule.departure_datetime
  Most queries filter flights by date, the absence of this index would result in a full collection scan across more than 4.7 million documents.
- Flight_info.flight_key
  Introduces near-constant-time retrieval for individual flight lookups.

Airport Collection

- Location.coordinates
  To enable efficient geospatial proximity searches of airports.

Find flights on a specific date

Test Scenario	Docs Examined	Docs Returned	Time (ms)
Without Index	4,772,506	18,175	49,344
With Index	18,175	18,175	618

Metrics about other indexes

Test Scenario	Docs Examined	Docs Returned	Time (ms)
flight_key	1	1	25
Geospatial Near 50km	5	4	144

Discussion on MongoDB Data Sharding

Flight Collection:
To handle heavy workloads (30k queries & 20k updates/day) the most effective solution using a Compound Shard Key is:

1. **Origin Airport (IATA Code):** Acts as the prefix to guarantee data locality. By grouping flights by their origin, the database can route the majority of search queries to specific, targeted shards. This avoids expensive operations across the cluster, reducing internal network traffic and optimizing read performance.

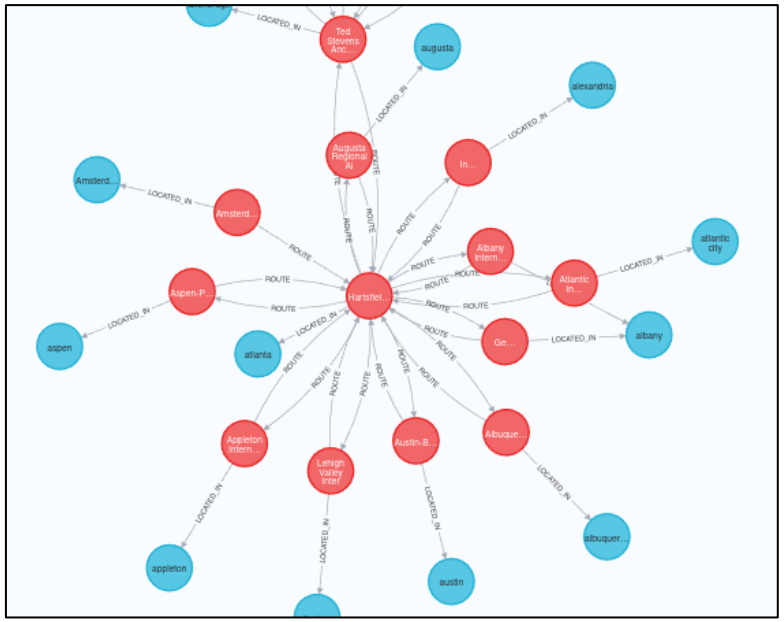
2. **Unique Flight ID:** Provides the essential granularity to prevent data hotspots at major, high-traffic hubs.
   Since every flight ID is unique, it allows MongoDB to split large chunks of data effectively, ensuring that heavy write loads, like continuous real-time flight logs updates, are evenly distributed across all available virtual machines.

The remaining collections do not require sharding due to their limited volume.

Graph DB Design

Nodes:
Airport: {iata_code, latitude, longitude, name}
City: {city_state, name, state_id, density, population, timezone}

Relationships:
Airport <- **Located_in** -> City : {}
Airport -> **Route** -> Airport : {mean_scheduled_time, num_flights}



Graph DB Design

Queries:

Best alternative airports ranked by similarity score

Domain-specific: find the best alternative airports within a 200km radius by prioritizing those that share the most flight destinations with the closed hub**.**

Graph-specific: find structurally similar nodes within a 200km spatial radius by ranking them based on the ratio of shared outgoing edges to their geographic distance from the closed hub**.**

```
@Query("MATCH (closed:Airport {iata_code: $closedIata})-[:ROUTE]->(dest:Airport) " +  1 usage  LGranucci
        "MATCH (alt:Airport)-[:ROUTE]->(dest) " +
        "WHERE alt.iata_code <> $closedIata " +
        "WITH closed, alt, count(DISTINCT dest) AS sharedConnections " +

        "WITH alt, sharedConnections, " +
        "     point.distance( " +
        "       point({latitude: closed.latitude, longitude: closed.longitude}), " +
        "       point({latitude: alt.latitude, longitude: alt.longitude}) " +
        "     ) / 1000.0 AS distKm " +
        "WHERE distKm > 0 AND distKm < 200 " +

        "RETURN alt.iata_code AS iata, alt.name AS name, (sharedConnections / log(distKm + 1)) AS score, '' as scoreType " +
        "ORDER BY score DESC " +
        "LIMIT 5")
List<AirportRankingDTO> findBestAlternativeAirports(@Param("closedIata") String closedIata);
```

Graph DB Design

Top hub ranking of airports

Domain-specific: find the top airport hubs by ranking them with a custom score that weighs direct outbound flights at 60% and indirect, one-stop connections at 40%.

Graph-specific: rank nodes using a custom centrality score based on the weighted sum of their immediate outgoing edges (60%) and their 1-hop neighbor edges (40%).

```
@Query("MATCH (airport:Airport)-[r1:ROUTE]->(dest:Airport) " +  1 usage  PaoloWalsh
        "WITH airport, sum(r1.num_voli) AS directTraffic " +

        "MATCH (airport)-[:ROUTE]->(dest:Airport)-[r2:ROUTE]->() " +
        "WITH airport, directTraffic, sum(r2.num_voli) AS indirectTraffic " +

        "RETURN airport.iata_code AS iata, " +
        "       airport.name AS name, " +
        "       (directTraffic * 0.6 + indirectTraffic * 0.4) AS score, '' AS scoreType " +
        "ORDER BY score DESC " +
        "LIMIT $limit")
List<AirportRankingDTO> findTopHubsByRank(@Param("limit") Integer limit);
```

Neo4j Indexes

Airport nodes:
Index on iata_code

City nodes:
Index on name

Execution	DB Accesses	Total Time (ms)
With Index	8	473
Without Index	630	512

Table 5.3: Data obtained from execution of "Get city by name" query

Execution	DB Accesses	Total Time (ms)
With Index	6	239
Without Index	690	298

Table 5.4: Data obtained from execution of "Get airport by iata_code" query

Handling Intra-DB consistency

Flight and Airport data CRUD:

The data shared between both databases includes is about airports and flights, to maintain data consistency the system updates MongoDB and Neo4j in a single operational flow.

Consistency:

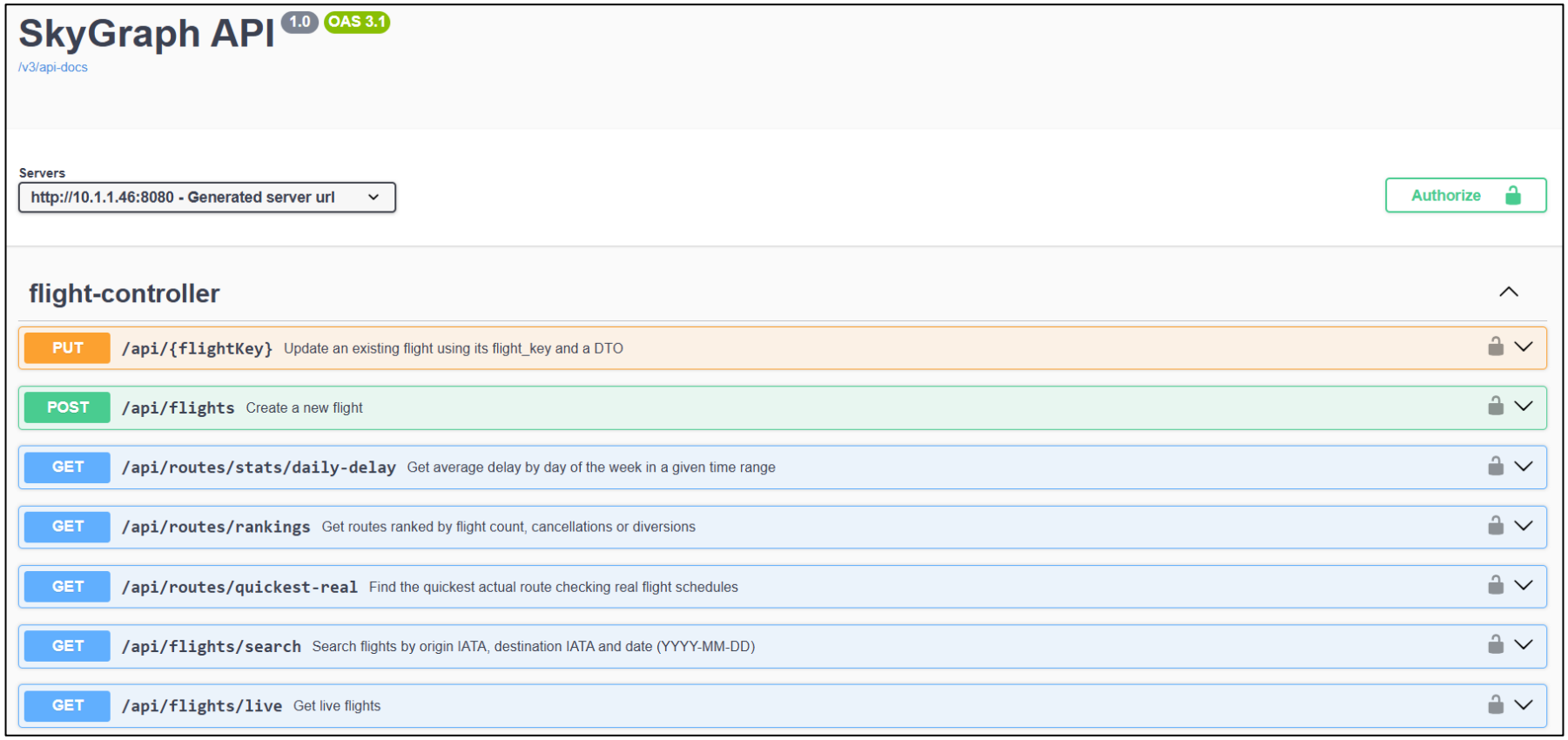
- **First:** write/update on Neo4j
  - If it fails: abort
- **Then:** write/update on MongoDB
  - If it fails: critical warning and manual rollback operation

```
try {
    airportRepository.upsertRouteRelationship(dto.getOriginIata(), dto.getDestinationIata(), dto.getDurationMinutes().doubleValue());
} catch (Exception e) {
    log.error("Neo4j creation failed for flightKey: {}. MongoDB insertion aborted.", flightKey, e);
    throw new RuntimeException("Neo4j Creation Failed: " + e.getMessage(), e);
}

try {
    return flightRepository.save(flightToSave);
} catch (Exception e) {
    log.error("POSSIBLE NEO4J INCONSISTENCY. Manual rollback needed for flightKey: {}. " +
                    "Action required: DECREMENT (remove) route {}->{} with duration {}.",
            flightKey, dto.getOriginIata(), dto.getDestinationIata(), dto.getDurationMinutes().doubleValue(), e);

    throw new RuntimeException("MongoDB Insertion Failed resulting in possible Neo4j inconsistency. Check system logs.", e);
}
```

Swagger UI REST APIs documentation



DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

UNIVERSITÀ DI PISA

CROSSLAB Innovation for industry 4.0